RAJALAKSHMI ENGINEERING COLLEGE

RAJALAKSHMI NAGAR, THANDALAM – 602 105



CS23331 - DESIGN AND ANALYSIS OF ALGORITHMS

Laboratory Record Note Book

Name KATHIRNILAVAN .G ……...

Year / Branch / Section : IInd YEAR – CSD A ……

University Register No. : 2116251701022 ……

College Roll No. : 251701022 ……

Semester : III SEM ……

Academic Year : 2025-2029 ……

INDEX PAGE

Exp.No	Name of the Experiment	Date of Completion
1	Finding Complexity using Counter Method	09-08-2026
2	Finding Complexity using Counter method	09-08-2026
3	Finding Complexity using Counter Method	09-08-2026
4	Finding Complexity using Counter Method	26-08-2026
5	Finding Complexity using counter method	26-08-2026
6	1-G-Coin Problem	09-09-2026
7	2-G-Cookies Problem	09-09-2026
8	3-G-Burger Problem	28-09-2026
9	4-G-Array Sum max problem	09-09-2026
10	5-G-Product of Array elements-Minimum	09-09-2026
11	Number of Zeros in a Given Array	30-09-2026
12	Majority Element	30-09-2026
13	Finding Floor Value	30-09-2026
14	Two Elements sum to x	30-09-2026
15	Implementation of Quick Sort	30-09-2026
16	DP-Playing with Numbers	07-10-2026
17	DP-Playing with chessboard	07-10-2026
18	DP-Longest Common Subsequence	07-10-2026
19	DP-Longest non-decreasing Subsequence	07-10-2026
20	Finding Duplicates-O(n^2) Time Complexity, O(1) Space Complexity	07-10-2026
21	Finding Duplicates-O(n) Time Complexity, O(1) Space Complexity	07-10-2026
22	Print Intersection of 2 sorted arrays-O(m*n) Time Complexity, O(1) Space Complexity	07-10-2026
23	Print Intersection of 2 sorted arrays-O(m+n) Time Complexity, O(1) Space Complexity	08-10-2026
24	Print Intersection of 2 sorted arrays-O(m+n) Time Complexity, O(1) Space Complexity	08-10-2026
25	Pair with Difference-O(n^2) Time Complexity, O(1) Space Complexity	08-10-2026
26	Pair with Difference-O(n) Time Complexity, O(1) Space Complexity	08-10-2026

RAJALAKSHMI ENGINEERING COLLEGE

Kathir Nilavan G 251701022-2025-CSD

K2

Started on	Monday, 28 September 2026, 3:34 PM
State	Finished
Completed on	Monday, 28 September 2026, 3:52 PM
Time taken	17 mins 45 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Problem Statement

Given an array of 1s and 0s this has all 1s first followed by all 0s. Aim is to find the number of 0s. Write a program using Divide and Conquer to Count the number of zeroes in the given array.

Input Format

First Line Contains Integer m – Size of array

Next m lines Contains m numbers – Elements of an array

Output Format

First Line Contains Integer – Number of zeroes present in the given array.

Answer: (penalty regime: 0 %)

Ace editor not ready. Perhaps reload page?

Falling back to raw text area.

```
#include <stdio.h>

int countZeroes(int arr[], int low, int high) {
    if (low > high)
        return 0;
    if (arr[low] == 0)
        return high - low + 1;
    if (arr[high] == 1)
        return 0;
    int mid = low + (high - low) / 2;
    return countZeroes(arr, low, mid) +
           countZeroes(arr, mid + 1, high);
}

int main() {
    int m;
    scanf("%d", &m);
```

	Input	Expected	Got	
✔	5 1 1 1 0 0	2	2	✔
✔	10 1 1 1 1 1 1 1 1 1 1	0	0	✔

	Input	Expected	Got	
✔	8 0 0 0 0 0 0 0 0	8	8	✔
✔	17 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0	2	2	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

RAJALAKSHMI ENGINEERING COLLEGE

Kathir Nilavan G 251701022-2025-CSD

K2

Started on	Tuesday, 29 September 2026, 7:28 PM
State	Finished
Completed on	Tuesday, 29 September 2026, 7:28 PM
Time taken	19 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Given an array `nums` of size `n`, return *the majority element.*

The majority element is the element that appears more than `⌊n / 2⌋` times. You may assume that the majority element always exists in the array.

Example 1:

```
Input: nums = [3,2,3]
Output: 3
```

Example 2:

```
Input: nums = [2,2,1,1,1,2,2]
Output: 2
```

Constraints:

- `n == nums.length`
- `1 <= n <= 5 * 10^4`
- `-2^31 <= nums[i] <= 2^31 - 1`

For example:

Input	Result
3 3 2 3	3
7 2 2 1 1 1 2 2	2

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

int main()
{
    int n;
    scanf("%d", &n);

    int nums[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &nums[i]);

    int candidate = nums[0];
    int count = 1;

    for (int i = 1; i < n; i++)
    {
        if (nums[i] == candidate)
            count++;
        else
            count--;

        if (count == 0)
        {
            candidate = nums[i];
            count = 1;
        }
    }

    printf("%d", candidate);

    return 0;
}
```

	Input	Expected	Got	
✔	3 3 2 3	3	3	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

Started on	Tuesday, 29 September 2026, 7:31 PM
State	Finished
Completed on	Tuesday, 29 September 2026, 7:33 PM
Time taken	1 min 5 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Problem Statement:

Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Value for x

Output Format

First Line Contains Integer – Floor value for x

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

int findFloor(int a[], int low, int high, int x)
{
    if (low > high)
        return -1;

    int mid = (low + high) / 2;

    if (a[mid] == x)
        return a[mid];

    if (a[mid] > x)
        return findFloor(a, low, mid - 1, x);

    int result = findFloor(a, mid + 1, high, x);

    if (result == -1)
        return a[mid];

    return result;
}

int main()
{
    int n, x;
    scanf("%d", &n);

    int a[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    scanf("%d", &x);

    printf("%d", findFloor(a, 0, n - 1, x));

    return 0;
}
```

	Input	Expected	Got	
✔	6 1 2 8 10 12 19 5	2	2	✔
✔	5 10 22 85 108 129 100	85	85	✔
✔	7 3 5 7 9 11 13 15 10	9	9	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Started on	Tuesday, 29 September 2026, 7:33 PM
State	Finished
Completed on	Tuesday, 29 September 2026, 7:33 PM
Time taken	30 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Problem Statement:

Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Value for x

Output Format

First Line Contains Integer – Floor value for x

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

int findFloor(int a[], int low, int high, int x)
{
    if (low > high)
        return -1;

    int mid = (low + high) / 2;

    if (a[mid] == x)
        return a[mid];

    if (a[mid] > x)
        return findFloor(a, low, mid - 1, x);

    int result = findFloor(a, mid + 1, high, x);

    if (result == -1)
        return a[mid];

    return result;
}

int main()
{
    int n, x;
    scanf("%d", &n);

    int a[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    scanf("%d", &x);

    printf("%d", findFloor(a, 0, n - 1, x));

    return 0;
}
```

	Input	Expected	Got	
✔	6 1 2 8 10 12 19 5	2	2	✔
✔	5 10 22 85 108 129 100	85	85	✔
✔	7 3 5 7 9 11 13 15 10	9	9	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Started on	Tuesday, 29 September 2026, 7:36 PM
State	Finished
Completed on	Tuesday, 29 September 2026, 7:41 PM
Time taken	4 mins 46 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Problem Statement:

Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as "No".

Note: Write a Divide and Conquer Solution

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Sum Value

Output Format

First Line Contains Integer – Element1

Second Line Contains Integer – Element2 (Element 1 and Elements 2 together sums to value "x")

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

int findPair(int a[], int left, int right, int x, int *e1, int *e2)
{
    if (left >= right)
        return 0;

    int sum = a[left] + a[right];

    if (sum == x)
    {
        *e1 = a[left];
        *e2 = a[right];
        return 1;
    }

    if (sum < x)
        return findPair(a, left + 1, right, x, e1, e2);

    return findPair(a, left, right - 1, x, e1, e2);
}

int main()
{
    int n, x, e1, e2;

    scanf("%d", &n);

    int a[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    scanf("%d", &x);

    if (findPair(a, 0, n - 1, x, &e1, &e2))
    {
        printf("%d\n", e1);
        printf("%d\n", e2);
    }
    else
    {
        printf("No");
    }

    return 0;
}
```

	Input	Expected	Got	
✔	4 2 4 8 10 14	4 10	4 10	✔
✔	5 2 4 6 8 10 100	No	No	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Started on	Tuesday, 29 September 2026, 7:43 PM
State	Finished
Completed on	Tuesday, 29 September 2026, 7:43 PM
Time taken	38 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Write a Program to Implement the Quick Sort Algorithm

Input Format:
The first line contains the no of elements in the list-n
The next n lines contain the elements.

Output:
Sorted list of elements

For example:

Input	Result
5 67 34 12 98 78	12 34 67 78 98

Answer:

```
    if (low < high)
    {
        int i = low;
        int j = high;
        int pivot = a[low];

        while (i < j)
        {
            while (a[i] <= pivot && i < high)
                i++;

            while (a[j] > pivot)
                j--;

            if (i < j)
            {
                int temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }
        }

        int temp = a[low];
        a[low] = a[j];
        a[j] = temp;

        quickSort(a, low, j - 1);
        quickSort(a, j + 1, high);
    }
}

int main()
{
    int n;
    scanf("%d", &n);

    int a[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    quickSort(a, 0, n - 1);

    for (int i = 0; i < n; i++)
    {
        if (i > 0)
            printf(" ");
        printf("%d", a[i]);
    }

    return 0;
}
```

	Input	Expected	Got	
✔	5 67 34 12 98 78	12 34 67 78 98	12 34 67 78 98	✔
✔	10 1 56 78 90 32 56 11 10 90 114	1 10 11 32 56 56 78 90 90 114	1 10 11 32 56 56 78 90 90 114	✔
✔	12 9 8 7 6 5 4 3 2 1 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Started on	Friday, 9 October 2026, 11:33 PM
State	Finished
Completed on	Friday, 9 October 2026, 11:34 PM
Time taken	1 min 24 secs
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 10.00 out of 10.00

Playing with Numbers:

Ram and Sita are playing with numbers by giving puzzles to each other. Now it was Ram term, so he gave Sita a positive integer 'n' and two numbers 1 and 3. He asked her to find the possible ways by which the number n can be represented using 1 and 3.Write any efficient algorithm to find the possible ways.

Example 1:

Input: *6*
Output:*6*
Explanation: *There are 6 ways to 6 represent number with 1 and 3*

1+1+1+1+1+1
3+3
1+1+1+3
1+1+3+1
1+3+1+1
3+1+1+1

Input Format

First Line contains the number n

Output Format

Print: The number of possible ways 'n' can be represented using 1 and 3

Sample Input

6

Sample Output

6

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    long long dp[n + 1];

    dp[0] = 1;

    for (int i = 1; i <= n; i++) {
        dp[i] = dp[i - 1];

        if (i >= 3) {
            dp[i] += dp[i - 3];
        }
    }

    printf("%lld\n", dp[n]);

    return 0;
}
```

	Input	Expected	Got	
✔	6	6	6	✔

	Input	Expected	Got	
✔	25	8641	8641	✔
✔	100	24382819596721629	24382819596721629	✔

Passed all tests! ✔

Correct

Marks for this submission: 10.00/10.00.

Started on	Friday, 9 October 2026, 11:40 PM
State	Finished
Completed on	Friday, 9 October 2026, 11:41 PM
Time taken	1 min 8 secs
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 10.00 out of 10.00

Playing with Chessboard:

Ram is given with an n*n chessboard with each cell with a monetary value. Ram stands at the (0,0), that the position of the top left white rook. He is been given a task to reach the bottom right black rook position (n-1, n-1) constrained that he needs to reach the position by traveling the maximum monetary path under the condition that he can only travel one step right or one step down the board. Help ram to achieve it by providing an efficient DP algorithm.

Example:

Input

3

1 2 4

2 3 4

8 7 1

Output:

19

Explanation:

Totally there will be 6 paths among that the optimal is

Optimal path value:1+2+8+7+1=19

Input Format

First Line contains the integer n

The next n lines contain the n*n chessboard values

Output Format

Print Maximum monetary value of the path

Answer: (penalty regime: 0 %)

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int a[n][n];
    int dp[n][n];

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &a[i][j]);
        }
    }

    dp[0][0] = a[0][0];

    for (int i = 1; i < n; i++) {
        dp[i][0] = dp[i - 1][0] + a[i][0];
    }

    for (int j = 1; j < n; j++) {
        dp[0][j] = dp[0][j - 1] + a[0][j];
    }

    for (int i = 1; i < n; i++) {
        for (int j = 1; j < n; j++) {
            int max = dp[i - 1][j] > dp[i][j - 1]
                      ? dp[i - 1][j]
                      : dp[i][j - 1];

            dp[i][j] = a[i][j] + max;
        }
    }

```

```
37      printf("%d\n", dp[n - 1][n - 1]);
38
39      return 0;
40  }
41
42
```

	Input	Expected	Got	
✔	3 1 2 4 2 3 4 8 7 1	19	19	✔
✔	3 1 3 1 1 5 1 4 2 1	12	12	✔
✔	4 1 1 3 4 1 5 7 8 2 3 4 6 1 6 9 0	28	28	✔

Passed all tests! ✔

Correct

Marks for this submission: 10.00/10.00.

Started on	Friday, 9 October 2026, 11:44 PM
State	Finished
Completed on	Friday, 9 October 2026, 11:44 PM
Time taken	45 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Given two strings find the length of the common longest subsequence(need not be contiguous) between the two.

Example:

s1: ggtabe

s2: tgatasb

s1		a	g	**g**	**t**	**a**	**b**
s2	**g**	x	**t**	x	**a**	y	**b**

The length is 4

Solveing it using Dynamic Programming

For example:

Input	Result
aab azb	2

Answer: (penalty regime: 0 %)

```
#include <stdio.h>
#include <string.h>

int main() {
    char s1[1000], s2[1000];
    int dp[1001][1001];

    scanf("%s", s1);
    scanf("%s", s2);

    int m = strlen(s1);
    int n = strlen(s2);

    for (int i = 0; i <= m; i++) {
        for (int j = 0; j <= n; j++) {
            if (i == 0 || j == 0) {
                dp[i][j] = 0;
            }
            else if (s1[i - 1] == s2[j - 1]) {
                dp[i][j] = dp[i - 1][j - 1] + 1;
            }
            else {
                dp[i][j] = dp[i - 1][j] > dp[i][j - 1]
                           ? dp[i - 1][j]
                           : dp[i][j - 1];
            }
        }
    }

    printf("%d\n", dp[m][n]);

    return 0;
}
```

	Input	Expected	Got	
✔	aab azb	2	2	✔
✔	ABCD ABCD	4	4	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Started on	Friday, 9 October 2026, 11:46 PM
State	Finished
Completed on	Friday, 9 October 2026, 11:48 PM
Time taken	1 min 23 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Problem statement:

Find the length of the Longest Non-decreasing Subsequence in a given Sequence.

Eg:

Input:9

Sequence:[-1,3,4,5,2,2,2,2,3]

the subsequence is [-1,2,2,2,2,3]

Output:6

Answer: (penalty regime: 0 %)

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int a[n], dp[n];

    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
        dp[i] = 1;
    }

    int max = 1;

    for (int i = 1; i < n; i++) {
        for (int j = 0; j < i; j++) {
            if (a[j] <= a[i] && dp[j] + 1 > dp[i]) {
                dp[i] = dp[j] + 1;
            }
        }

        if (dp[i] > max) {
            max = dp[i];
        }
    }

    printf("%d\n", max);

    return 0;
}
```

	Input	Expected	Got	
✔	9 -1 3 4 5 2 2 2 2 3	6	6	✔
✔	7 1 2 2 4 5 7 6	6	6	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Started on	Friday, 9 October 2026, 11:50 PM
State	Finished
Completed on	Friday, 9 October 2026, 11:51 PM
Time taken	38 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Find Duplicate in Array.

Given a read only array of n integers between 1 and n, find one number that repeats.

Input Format:

First Line - Number of elements

n Lines - n Elements

Output Format:

Element x - That is repeated

For example:

Input	Result
5 1 1 2 3 4	1

Answer: (penalty regime: 0 %)

```c

#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int a[n];

    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (a[i] == a[j]) {
                printf("%d\n", a[i]);
                return 0;
            }
        }
    }

    return 0;
}

```

	Input	Expected	Got	
✔	11 10 9 7 6 5 1 2 3 8 4 7	7	7	✔
✔	5 1 2 3 4 4	4	4	✔
✔	5 1 1 2 3 4	1	1	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Started on	Friday, 9 October 2026, 11:53 PM
State	Finished
Completed on	Friday, 9 October 2026, 11:54 PM
Time taken	40 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Find Duplicate in Array.

Given a read only array of n integers between 1 and n, find one number that repeats.

Input Format:

First Line - Number of elements

n Lines - n Elements

Output Format:

Element x - That is repeated

For example:

Input	Result
5 1 1 2 3 4	1

Answer: (penalty regime: 0 %)

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int a[n];

    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (a[i] == a[j]) {
                printf("%d", a[i]);
                return 0;
            }
        }
    }

    return 0;
}
```

	Input	Expected	Got	
✔	11 10 9 7 6 5 1 2 3 8 4 7	7	7	✔
✔	5 1 2 3 4 4	4	4	✔
✔	5 1 1 2 3 4	1	1	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Started on	Friday, 9 October 2026, 11:56 PM
State	Finished
Completed on	Friday, 9 October 2026, 11:57 PM
Time taken	1 min 19 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Find the intersection of two sorted arrays.

OR in other words,

Given 2 sorted arrays, find all the elements which occur in both the arrays.

Input Format

· The first line contains T, the number of test cases. Following T lines contain:

1. Line 1 contains N1, followed by N1 integers of the first array
2. Line 2 contains N2, followed by N2 integers of the second array

Output Format

The intersection of the arrays in a single line

Example

Input:

1

3 10 17 57

6 2 7 10 15 57 246

Output:

10 57

Input:

1

6 1 2 3 4 5 6

2 1 6

Output:

1 6

For example:

Input	Result
1 3 10 17 57 6 2 7 10 15 57 246	10 57

Answer: (penalty regime: 0 %)

```
#include <stdio.h>

int main() {
    int T;
    scanf("%d", &T);

    while (T--) {
        int n1, n2;
        scanf("%d", &n1);

        int a[n1];
        for (int i = 0; i < n1; i++) {
            scanf("%d", &a[i]);
        }

        scanf("%d", &n2);

        int b[n2];
        for (int i = 0; i < n2; i++) {
            scanf("%d", &b[i]);
        }
```

```
        int i = 0, j = 0, first = 1;

        while (i < n1 && j < n2) {
            if (a[i] == b[j]) {
                if (!first) {
                    printf(" ");
                }
                printf("%d", a[i]);
                first = 0;
                i++;
                j++;
            }
            else if (a[i] < b[j]) {
                i++;
            }
            else {
                j++;
            }
        }

        printf("\n");
    }

    return 0;
}
```

	Input	Expected	Got	
✔	1 3 10 17 57 6 2 7 10 15 57 246	10 57	10 57	✔
✔	1 6 1 2 3 4 5 6 2 1 6	1 6	1 6	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Started on	Saturday, 10 October 2026, 12:00 AM
State	Finished
Completed on	Saturday, 10 October 2026, 12:00 AM
Time taken	49 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Find the intersection of two sorted arrays.

OR in other words,

Given 2 sorted arrays, find all the elements which occur in both the arrays.

Input Format

· The first line contains T, the number of test cases. Following T lines contain:

1. Line 1 contains N1, followed by N1 integers of the first array
2. Line 2 contains N2, followed by N2 integers of the second array

Output Format

The intersection of the arrays in a single line

Example

Input:

1

3 10 17 57

6 2 7 10 15 57 246

Output:

10 57

Input:

1

6 1 2 3 4 5 6

2 1 6

Output:

1 6

For example:

Input	Result
1 3 10 17 57 6 2 7 10 15 57 246	10 57

Answer: (penalty regime: 0 %)

```
#include <stdio.h>

int main() {
    int T;
    scanf("%d", &T);

    while (T--) {
        int n1, n2;
        scanf("%d", &n1);

        int a[n1];
        for (int i = 0; i < n1; i++) {
            scanf("%d", &a[i]);
        }

        scanf("%d", &n2);

        int b[n2];
        for (int i = 0; i < n2; i++) {
            scanf("%d", &b[i]);
        }
```

```
        }

        int i = 0, j = 0, first = 1;

        while (i < n1 && j < n2) {
            if (a[i] == b[j]) {
                if (!first)
                    printf(" ");

                printf("%d", a[i]);
                first = 0;
                i++;
                j++;
            }
            else if (a[i] < b[j]) {
                i++;
            }
            else {
                j++;
            }
        }

        printf("\n");
    }

    return 0;
}
```

	Input	Expected	Got	
✔	1 3 10 17 57 6 2 7 10 15 57 246	10 57	10 57	✔
✔	1 6 1 2 3 4 5 6 2 1 6	1 6	1 6	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Started on	Saturday, 10 October 2026, 12:02 AM
State	Finished
Completed on	Saturday, 10 October 2026, 12:03 AM
Time taken	41 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Given an array A of sorted integers and another non negative integer k, find if there exists 2 indices i and j such that A[j] - A[i] = k, i != j.

Input Format:

First Line n - Number of elements in an array

Next n Lines - N elements in the array

k - Non - Negative Integer

Output Format:

1 - If pair exists

0 - If no pair exists

Explanation for the given Sample Testcase:

YES as 5 - 1 = 4

So Return 1.

For example:

Input	Result
3 1 3 5 4	1

Answer: (penalty regime: 0 %)

```
#include <stdio.h>

int main() {
    int n, k;
    scanf("%d", &n);

    int a[n];

    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    scanf("%d", &k);

    int i = 0, j = 1;

    while (i < n && j < n) {
        int diff = a[j] - a[i];

        if (diff == k && i != j) {
            printf("1\n");
            return 0;
        }
        else if (diff < k) {
            j++;
        }
        else {
            i++;
        }

        if (i == j) {
            j++;
        }
    }

    printf("0\n");

    return 0;
}
```

```
42
```

	Input	Expected	Got	
✔	3 1 3 5 4	1	1	✔
✔	10 1 4 6 8 12 14 15 20 21 25 1	1	1	✔
✔	10 1 2 3 5 11 14 16 24 28 29 0	0	0	✔
✔	10 0 2 3 7 13 14 15 20 24 25 10	1	1	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Started on	Saturday, 10 October 2026, 12:07 AM
State	Finished
Completed on	Saturday, 10 October 2026, 12:08 AM
Time taken	1 min 34 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Given an array A of sorted integers and another non negative integer k, find if there exists 2 indices i and j such that A[j] - A[i] = k, i != j.

Input Format:

First Line n - Number of elements in an array

Next n Lines - N elements in the array

k - Non - Negative Integer

Output Format:

1 - If pair exists

0 - If no pair exists

Explanation for the given Sample Testcase:

YES as 5 - 1 = 4

So Return 1.

For example:

Input	Result
3 1 3 5 4	1

Answer: (penalty regime: 0 %)

```
#include <stdio.h>

int main() {
    int n, k;
    scanf("%d", &n);

    int a[n];

    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    scanf("%d", &k);

    int i = 0, j = 1;

    while (i < n && j < n) {
        if (i == j) {
            j++;
            continue;
        }

        int diff = a[j] - a[i];

        if (diff == k) {
            printf("1\n");
            return 0;
        }
        else if (diff < k) {
            j++;
        }
        else {
            i++;
        }
    }

    printf("0\n");

    return 0;
}
```

```
42
43
```

	Input	Expected	Got	
✔	3 1 3 5 4	1	1	✔
✔	10 1 4 6 8 12 14 15 20 21 25 1	1	1	✔
✔	10 1 2 3 5 11 14 16 24 28 29 0	0	0	✔
✔	10 0 2 3 7 13 14 15 20 24 25 10	1	1	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.